CHAPTER 8

LANGUAGES

Contents:

1. Strings and Languages
2. Finite Specification of Languages
3. Regular Sets and Expressions

Strings and Languages

- Type of languages: Natural languages, computer languages, mathematical languages
- In CS, a language is a set of strings over an alphabet.
- Syntax of the language: certain properties that must be satisfied by strings.
- Strings of characters are fundamental building blocks in computer science.

Symbol, Alphabets, String

- Symbols or Letters or Characters:

  A single element of the alphabets that has a unique meaning, i.e.: symbol A and B which have different meanings.

- Alphabet (denoted by Σ in italic capital letters):

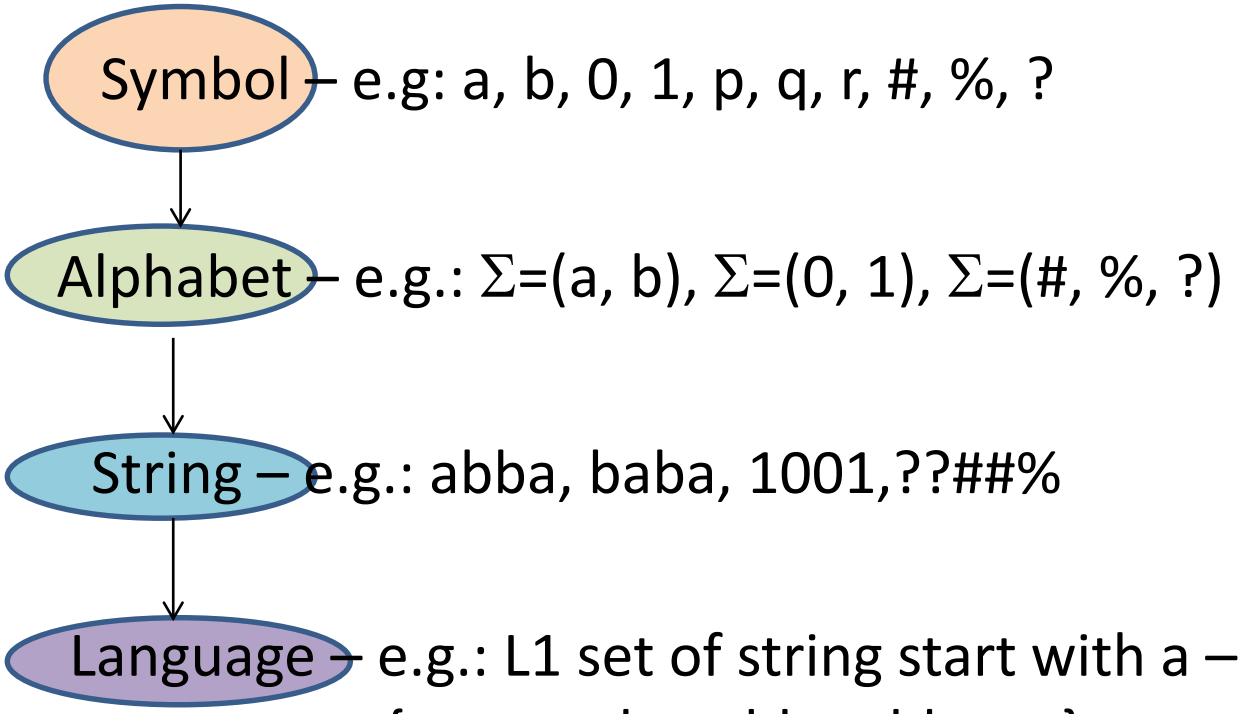
  A finite set of symbols – indivisible objects.

- String or Word:

  A finite sequence of symbols from alphabets.

- A language is a set of strings over an alphabet.

Example


Symbol – e.g: a, b, 0, 1, p, q, r, #, %, ?
Alphabet – e.g.: Σ=(a, b), Σ=(0, 1), Σ=(#, %, ?)
String – e.g.: abba, baba, 1001,??##%
Language – e.g.: L1 set of string start with a –
{a, aa, aab, aabb,aabba, …}

Strings and Alphabets

- Example: C = {a, b, c, 1, 2, 3}

✓Alphabet C with 6 units of symbols.

An example of a word or string from the alphabet C :
acca, baca, 132, a12, etc.

✓String acca and caac have different meanings.

✓String acca, 121, abba are palindromes.

*palindromes = a word, phrase, or sequence that reads the same backwards as forwards.

Strings and Alphabets

- String over an alphabet is a finite sequence of symbols from that alphabet, written next to one another and not separated by commas.
- Example:

  Let $\Sigma = \{0,1\}$, then strings over Σ are:

  0, 1, 00, 01, 10, 11, …

  1010, …

  1110100111, …

  1111111111, … and etc.

Strings and Alphabets

Length of String

- If w is a string over Σ, the length of w, written $|w|$, is the number of symbols that it contains.
- Example:

$$
\begin{aligned}
|\lambda| &= 0 \\
|0| &= 1 \\
|1| &= 1 \\
|1010| &= 4 \\
|001101| &= 6
\end{aligned}
$$

Strings and Alphabets

- For alphabet $\Sigma = \{a, b, c\}$, Σ^* is the set of all strings over Σ. It includes:

Σ^0 = length 0:	λ or ε
Σ^1 = length 1:	a b c
Σ^2 = length 2:	aa ab ac ba bb bc ca cb cc
Σ^3 = length 3:	aaa aab aac aba abb abc aca acb acc baa bab bac bba bbb bbc bca bcb bcc caa cab cac cba cbb cbc cca ccb ccc
...	...
Σ^n = set of all strings of length *n*.	
:	:

Strings and Language

- A *language* over Σ is a set of string, $L \subseteq \Sigma^*$.
- Example:

  L = {1, 01, 11, 001} is a language over {0, 1}.
- Let, *L2*= {00, 01, 10, 11}
  - Language L^2: a set of string over alphabet {0, 1} of length 2.
  - *L2* is finite language.
- Let,*L3* = {0, 00, 01, 000, 001, 010, 011, ….}
  - Language*L3*: a set of string over alphabet {0, 1} start with 0.
  - *L3* is infinite language.

Exercise

Σ = { P, Q, R}

$L2$= set of string that has length 2. Write the example of string for$L2$.

String Recursive Definition

Let $\sum$ be an alphabet.

The set of strings over $\sum$ (written as $\sum^*$) is defined recursively as follows:

- Basis : $\lambda \in \sum^*$
- Recursive step: if $w \in \sum^*$ and $a \in \sum$

  Then, $wa \in \sum^*$
- Closure: $w \in \sum^*$ if and only if it can be obtained from basis element λ by a finite number of applications of recursive step.

Concatenation of String

- If we have string x of length m and string y of length n, the concatenation of x and y written xy is $x_1 \ldots x_m y_1 \ldots y_n$.
- Example:

x = aba; y = bbbab

Then, xy = ababbbab

yx = bbbabaaba

xyx = ababbbabaaba

Self Concatenation

- If we have string x, the concatenation of x and x is self concatenation.
- Example:

$x^0 = \lambda$

$x^1 = x = aba$

$x^2 = xx = abaaba$

$x^3 = xxx = abaabaaba$

- $x^k = xx...x$: self-concatenated string *k* times

Definition of Concatenation

Let $u, v \in \sum^*$.

The concatenation of u and v, written as uv,

is a binary operation on $\sum^*$ defined as follows:

- Basis : length (v)=0, then $v=\lambda$ and uv=u
- Recursive step:

  Let v be a string with length(v)=n>0.

  Then v=wa, and uv=(uw)a

  for some string w with length n-1 and $a \in \sum$

Concatenation

Let u = ab, v = ca, w = bb.

Then the concatenation of:

uv = abca

vw = cabb

(uv)w = abcabb

u(vw) = abcabb

The result is independent of the order in which the operations are performed.

Substring

- String z is a substring of w if z appears consecutively within w (z occurs inside of w)
- Thus, substring is a subset of the symbols in a string.
- Example:

  w = abbaaababb

  bba is a substring of w

  abab is a substring of w

  baba is NOT a substring of w

Exercise

Determine whether the following is substring or not for **Q = abbaaababb**

a) aab

b) abab

c) bbb

d) baba

Prefix

- Prefix for string v is a substring u where :

  if $x = \lambda$, for $v = xuy$

  Then $v = uy$ (string v starts with substring u)
- A prefix is any sequence of leading symbols of the string.
- Example:

  If w = abbaaababb

  a is a prefix of w

  abbaa is a prefix of w

  bba is NOT a prefix of w

Suffix

- Suffix for string v is a substring u where :

  if $y = \lambda$, for $v = xuy$

  Then, $v = xu$ (string v ends with substring u)
- A suffix is any sequence of trailing symbols of the string.
- Example:

  If w = abbaaababb

  abb is a suffix of w

  babb is a suffix of w

  bab is NOT a suffix of w

Reverse

- The reverse of w, written w^R or w^r is the string obtained by writing w in the opposite order.
- Example:

if w = a;	w^R = a
if w = abb;	w^R = bba
if w = aba;	w^R = aba
if w = abbcd;	w^R = dcbba

Finite Spec. of Languages

- Languages can be defined using 2 ways. It either presented as:
  i. an alphabet and the exhaustive list of all valid words, or
  ii. an alphabet and a set of rules defining the acceptable words.

PALINDROME Language

PALINDROME = {ε, and all string x such that reverse(x) = x}

- Example:

  Definition of a new language PALINDROME over the alphabet: $\Sigma = \{a, b\}$ will give a listing (valid):

  PALINDROME = {ε, a, b, aa, bb, aaa, aba, bab, bbb, aaaa, abba…}

- If we concatenate 2 words in PALINDROME, sometimes it can produce a new words which is also in PALINDROME but sometimes it doesn't.

* ε = empty string

Kleene Closure/Star

- Σ^* is the collection of all possible finite-length strings generated from the strings in Σ.
- The definition of Kleene star on Σ is

$$\Sigma^* = \bigcup_{i \in \mathbb{N}} \Sigma^i{}_i = \{\lambda\} \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \ldots$$

Kleene Closure/Star

- Example: If $\Sigma = \{x\}$, then $\Sigma^* = \{x\}^* = \{\varepsilon\ x\ xx\ xxx\ldots\}$

  If $\Sigma = \{0\ \ 1\}$, then

  $\Sigma^* = \{\varepsilon\ \ 0\ \ 1\ \ 00\ \ 01\ \ 10\ \ 11\ \ 000\ \ 001\ldots\}$

- Applied to set of strings:

  {ab, c}* = {λ, ab, c, abab, abc, cab, cc, ababab, ababc, abcab, abcc, cabab, cabc, ccab, ccc, ...}

Exercise

If $\Sigma = \{1, ab\}$, write the strings for the following:

a) $\Sigma^0 = \lambda$

b) $\Sigma^1 = \{$

c) $\Sigma^2 = \{$

d) $\Sigma^3 = \{$

Kleene Closure/Star

- Applied to set of characters:

  {a, b, c}* = {λ, a, b, c, aa, ab, ac, ba, bb, bc, ca, cb, cc, ...}.

- Kleene star applied to the empty set:

$$\varnothing^* = \{\lambda\}.$$

Kleene Closure/Star

- We can think of the Kleene star as:

  An operation that makes an infinite language of strings of letter out of an alphabet.

  Infinite language = infinitely many words, each of finite length.

Definition of S*

- If S is a set of words, then by S* :

  Means the set of all finite strings formed by concatenating words from S,

  where any word may be used as often as we like, and where the null string is also included.

Example

- If S = {aa, b}, then

  S* = {λ plus any word composed of

  factors of aa and b}

  = {λ plus all strings of a's and b's in

  which the a's occur in even clumps}

  = {λ b aa bb aab baa bbb … }

Example

- $S^* = \{aa\ \ b\}^*$, where * = 0, 1, 2, 3, …

$\{aa\ \ b\}^0 = \{\lambda\}$

$\{aa\ \ b\}^1 = \{aa, b\}$

$\{aa\ \ b\}^2 = \{aa, b\}\{aa, b\}$

$= \{aaaa, aab, baa, bb\}$

$\{aa\ \ b\}^3 = \{aa, b\}\{aa, b\}\{aa, b\}$

$= \{aaaaaa, aaaab, aabaa, aabb, baaaa, baab, bbaa, bbb\}$

$\{aa\ \ b\}^4 = \{aa\ b\}\{aa\ b\}\{aa\ b\}\{aa\ b\}$

$= \{aaaaaaaa, aaaaaab, aaaabaa, aaaabb, aabaaaa, aabaab, aabbaa, aabbb, baaaaaa, baaaab, baabaa, baabb, bbaaaa, bbaab, bbbaa, bbbb\}$

……..

Example

- If S = {a ab}, then

S* = {λ plus any word composed of factors of a and ab}

= {λ plus all strings of a's and b's except those that start with b and those that contain a double b}

= {λ a aa ab aaa aab aba … }

Example

- Consider the following languages

$$S = \{a \;\; b \;\; ab\} \quad \text{and} \quad T = \{a \;\; b \;\; bb\}$$

both S^* and T^* are languages of all strings of a's and b's since any string of a's and b's can be factored into syllables of either (a) or (b), both of which are in S and T.

Kleene Plus / Positive Closure $^+$

- If we would like to refer to only the concatenation of some (not zero) strings from a set S, we use the notation + instead of *,
- Example: If $\Sigma = \{x\}$, then

$$\Sigma^+ = \{x \ xx \ xxx \ldots\}$$

- Kleene plus applied to the empty set:

$$\varnothing^+ = \varnothing\varnothing^* = \{\} = \varnothing.$$

S^+

- If S is a set of strings not include λ, then S^+ is the language S^* without the word λ.
- If S is a language that does contain λ, then $S^+ = S^*$.
- S^+ can contain λ only when S contains the word ε initially.

S* and S**

Theorem 1:

For any set S of strings, we have $S^* = S^{**}$

Proof:

Every word in S^{**} is made up of factors from S^*. Every factor from S^* is made up of factors from S. Therefore, every word in S^{**} is made up of factors from S. Therefore, every word in S^{**} is also a word in S^*.

$S^{**} \subset S^*$ (is contained in or equal to)

Regular Expressions (RegEx)

- RegEx is rules that define exactly the set of words that are valid in a formal language.
- Formally, the set of regular expressions can be defined by the following recursive rules:
  1. Every symbol of Σ is a regular expression.
  2. ε is a regular expression.
  3. If R1 and R2 are regular expressions, so are

     (R1) R1R2 R1 | R2 R1*
  4. Nothing else is a regular expression.

Regular Expressions (RegEx)

- Regular expressions are defined recursively
  a) Base case – simple regular expressions
  b) Recursive case – how to build more complex regular expressions from simple regular expressions

Formal Definition of Regular Expressions

- A language is regular if it can be described by a regular expression.
- The Regular Languages is the set of all languages that can be represented by a regular expression.
  - Set of set of strings.
- Not every languages are able to be described by RegEx. Regular languages may also be described by another fine definitions, besides the RegEx.

Regular Expressions (RegEx)

- The symbols that appear in RegEx are:
  - the letters of the alphabet Σ
  - the symbol of null string ε or λ
  - parentheses ( )
  - star operator *
  - $\cup$ or + sign

Regular Operations

- Let A and B be languages. We define the regular operations union, concatenation, and star as follows.
  - Union: $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$
  - Concatenation: (simply no written)

    $$A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$$
  - Star: (repetition)

    $$A^* = \{x_1x_2x_3 \ldots x_k \mid k \geq 0 \text{ and each } x_i \in A\}$$

Union (∪)

$x \cup y$ where x and y are strings of characters from an alphabet means,

"either x or y"

Also written as $x + y$

Example

Let Σ = {a, b, c…z}, if A = {care, harm}, B = {ful, less}

Union:

A U B = (A + B)= {care, harm, ful, less}

Concatenation:

AB = {careful, careless, harmful, harmless}

Star / repetition:

B* = {λ, ful, less, fulful, fuless, lessful, lessless,…}

A* = {λ, care, harm, carecare, careharm, harmcare harmharm, …}

(A U B)* = {λ, care, harm, ful, less, carecare, careharm, careful, careless, harmcare, harmharm, harmful, harmless, …}

IN or NOT IN?

- ab*

  IN a, ab, abb, abbb, abbbb,

  NOT IN b, ba, aba, abab, bab, bbb, aab, baa, abba

- (ab)*

  IN λ, ab, abab, ababab, abababab,

  NOT IN b, ba, bb, abb, baa, bba, bab, bbb

- (a*b*)

  IN λ, a, b, aa, ab, bb, aaa, aab, abb, bbb, aaaa

  NOT IN ba, bab, abab, bba, aba, bbba

Regular Expressions (RegEx)

- We shall develop some new language-definition symbolism that will be much more precise than the …

Example:

$$L_1 = \{\varepsilon \;\; x \;\; xx \;\; xxx \;\; xxxx \ldots\}$$

We can define it with closure.

Let, $S = \{x\}$; then $L_1 = S^*$

or we can write $L_1 = \{x\}^*$

Language (x*)

- We can also define L_2 as

$$L_2 = (x^*)$$

Since x* is any string of x's, L_2 is then the set of all possible string of x's of any length (including λ).

Example

- Suppose we wish to describe the language L over the alphabet

$$\Sigma = \{a \;\; b\}$$

where $L = \{a \;\; ab \;\; abb \; abbb \; abbbb \; \ldots\}$

"all words of the form one a followed by some number of b's (maybe no b's at all)"

we may write, $L = (ab^*)$

(ab)*

(ab)* = ε or ab or abab or ababab …

- Parentheses are not letters in the alphabet of this language, so they can be used to indicate factoring without accidentally changing the words.
- Like the powers in algebra

ab* means a(b*), not (ab)*

(xx^*) vs. (x^+)

$$L_1 = (xx^*)$$

This mean we start each word of L_1 by writing down an x and then we follow it with some string of x's (which may be no more x's at all.)

We can use the $^+$ notation and write,

$$L_1 = (x^+)$$

$L_1 = (xx^*)$ and $L_1 = (x^+)$

- The language L_1 defined above can also be defined by any of these expressions:

xx^*x^+ xx^*x^* x^*xx^*

x^+x^* x^*x^+ $x^*x^*x^*xx^*$

Remember,

x^* can always be λ

Example

ab*a

is the set of all string of a's and b's that have at least two letters, that begin and end with a's, and that have nothing but b's inside (if anything at all).

(ab*a) = {aa aba abba abbba abbbba …}

Example

a*b*

Contains all the strings of a's and b's in which all the a's (if any) come before all b's (if any)

(a*b*) = {ε a b aa ab bb aaa aab abb bbb aaaa …}

notice that,

ba and aba are not in this language.

a*b* vs. (ab)*

$$a^*b^* \neq (ab)^*$$

(ab)* can contain abab

but

a*b* can't contain abab

Example

- Consider the language T defined over the alphabet $\Sigma = \{a\ b\ c\}$

  $T = \{a\ c\ ab\ cb\ abb\ cbb\ abbb\ cbbb\ abbbb\ cbbbb\ldots\}$

  all the words in T begin with an a or a c and then are followed by some number of b's.

  $T = ((a \cup c)\ b^*)$

  = (either a or c then some b's)

Finite Language L

- We can define any finite language by our new expression.
- Example: Consider a finite language L contains all the strings of a's and b's of length 3 exactly:

  $L = \{aaa\ aab\ aba\ abb\ baa\ bab\ bba\ bbb\}$

- The first letter can be either a or b. So do the 2nd and 3rd letter.

  $L = ((a \cup b)\ (a \cup b)\ (a \cup b))$

Finite language (cont.)

or we can simply write shortly as

$$L = (a \cup b)^3$$

if we write $(a \cup b)^*$, it means the set of all possible strings of letters from the alphabet $\Sigma = \{a\ b\ c\}$

including the null string λ.

Examples

- If we write

$$a(a \cup b)^*$$

we can describe all words that begin with the letter a.

- If we would like to describe all words that begin with an a and end with b, we can define by the expression

$$a(a \cup b)^*b = a(\text{arbitrary string})b$$

Null Language

- ε or λ is the symbol of null string in regular expression.
- $\varnothing$ is the symbol for "Null Language"
- Don't confuse!
  - $R = \lambda$ represents the language containing a single string, the empty string. ⇨ $\{\lambda\}$
  - $R = \varnothing$ represents the language that doesn't contain any strings.

Example

- Let consider the language defined by

$$(a \cup b)^*a(a \cup b)^*$$

- What does it produce ?

  The language of all words over the alphabet Σ = {a, b} that have an *a* in somewhere.

  Only words which are not in this language are those that have only b's and the word ε.

Union of Two Languages

- Those words which compose of only b's are defined by the expression b*.

  (b* also includes the null string, ε)

- Therefore, the language of all strings over the alphabet Σ = {a, b} are:

  all strings = (all strings with an a) ∪ (all string without an a)

$$(a \cup b)^* = (a \cup b)^* a (a \cup b)^* \cup b^*$$

Example

- How can we describe the language of all words that have at least two a's ?

Answer:

$$(a \cup b)^*a(a \cup b)^*a(a \cup b)^*$$

= (some beginning)(the first a)(some middle)(the second a)(some end)

where the arbitrary parts can have as many a's (or b's) as they want.

Example

- Is there any other RegEx that can define the language with at least two a's ?

Answer: Yes. For example: $b^*ab^*a(a \cup b)^*$

= (some beginning of b's (if any))(the first a) (some middle of b's)(the second a) (some end)

Equivalent Expressions

$(a \cup b)^* a\ (a \cup b)^* a\ (a \cup b)^* = b^* a b^* a (a \cup b)^*$

Both expressions are equivalent because they both describe the same item. We could write

$((a \cup b)^* a\ (a \cup b)^* a\ (a \cup b)^*)$

$= (b^* a b^* a (a \cup b)^*)$

= all words with at least two a's

$= (a \cup b)^* a b^* a b^*$

$= b^* a (a \cup b)^*\ a b^*$

Example

- If we wanted all words with exactly two a's, we could use the expression:

$$b^*ab^*ab^*$$

it can describe such words as:

aab, baba, bbbabbbab, …

Question: Can it make the word aab ?

Answer: Yes, by having the first and second

$$b^* = \lambda$$

Example

- How about the language with at least one a and at least one b ?

$$(a \cup b)^* a (a \cup b)^* b (a \cup b)^*$$

It can only produce words which an a precede a b. To produce words which have a b precede an a, we can describe by

$$(a \cup b)^* b (a \cup b)^* a (a \cup b)^*$$

Thus, the set of all words :

$$(a \cup b)^* a\ (a \cup b)^* b\ (a \cup b)^* \cup\ (a \cup b)^* b\ (a \cup b)^* a\ (a \cup b)^*$$

Example

- $(a \cup b)^* a\ (a \cup b)^* b\ (a \cup b)^*$ can produce all words with at least one a and at least one b.
- However, it doesn't contain the words of the forms some b's followed by some a's.
- These exceptions are all defined by bb^*aa^*
- Thus, we have all strings over $\Sigma = \{a, b\}$

$$(a \cup b)^* a\ (a \cup b)^* b\ (a \cup b)^* \cup\ (a \cup b)^* b\ (a \cup b)^* a\ (a \cup b)^*$$

$$= (a \cup b)^* a\ (a \cup b)^* b\ (a \cup b)^* \cup\ bb^*aa^*$$

(a∪b)*

(a∪b)*a (a∪b)*b (a∪b)*∪ bb*aa*

- generates all words which have both a and b in them somewhere.
- Words which are not included in the above expression are words of all a's, all b's or ε ⇨ a*, b*
- Now, we have all words which can be generated above the alphabet

(a∪b)* = (a∪b)*a (a∪b)*b (a∪b)*∪ bb*aa* ∪ a* ∪ b*

Example

What is the regular expression for the following language:

- All strings over {0, 1} that contain 101

  Ans: (0+1)*101(0+1)*

- All strings over {0, 1} that do not contain 01

  Ans: 1*0*

- All strings over {0, 1} with no occurrences of 00

  Ans: 1*(011*)*(0+1*)

- All strings over {0, 1} with exactly one occurrence of 00

  Ans: 1*(011*)*00(11*0)*1*

Regular Expressions (RegEx)

Write the example of string for the following RegEx:

i. (a + b)

ii. (a + b)*

iii. (((a + b)(b*)) a)

iv. ((a ((a + b)*)) a)

v. ((a*)(b*))

vi. ((ab)*)

Regular Expressions (RegEx)

i. (a + b) = {a, b}

ii. (a + b)* = {λ, a, b, aa, ab, ba, bb, aaa, aab, abb…}

iii. (((a + b)(b*)) a) = (a + b)b*a

{aa, ba, aba, bba, abba, bbba, abbba, bbbba, . . .}

iv. ((a ((a + b)*)) a) = a(a+b)*a

{aa, aaa, aba, aaaa, aaba, abaa, abba, . . .}

v. ((a*)(b*)) = a*b*

{λ, a, b, aa, ab, bb, aaa, aab, abb, bbb, . . .}

vi. ((ab)*) = (ab)*

{λ, ab, abab, abababab, ababababab, . . .}

Note

- Sets $\phi = \{\ \} \neq \{\lambda\}$
- Set size $|\{\ \}| = |\phi| = 0$
- Set size $|\{\lambda\}| = 1$
- String length $|\lambda| = 0$